calibration.py — technical

`src/moe_pruning/data/calibration.py · 212 lines · moe_pruning.data.calibration`

Purpose

Reads the `human`/`response` JSONL, tokenizes it through the model's task, and defines the deterministic train/validation split used by every later stage.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class CalibrationDataError`

lines 17-18 · bases ValueError

Raised when the calibration file is missing or has no valid records.

`class CalibrationRecord`

lines 22-25 · dataclass

One validated calibration pair (human, response, line number).

Field	Type	Default
`human`	str	
`response`	str	
`line_number`	int	

`class CalibrationLoadReport`

lines 29-47 · dataclass

Counts of loaded and skipped lines plus the first problems found.

Field	Type	Default
`path`	str	
`lines_read`	int	0
`loaded`	int	0
`skipped_blank_lines`	int	0
`skipped_malformed_json`	int	0
`skipped_missing_fields`	int	0
`skipped_empty_fields`	int	0
`truncated_at_max_samples`	bool	False
`problems`	list[str]	field(default_factory=list)

`CalibrationLoadReport.problem`

```
def problem(self, message: str) -> None
```

lines 42-44

Remember a problem message (the first 20 are kept).

`CalibrationLoadReport.to_dict`

```
def to_dict(self) -> dict
```

lines 46-47

Convert to a plain dictionary for JSON reports.

load_calibration_records

```
def load_calibration_records(path: Path, max_samples: int | None=None) -> tuple[list[CalibrationRecord], CalibrationLoadReport]
```

lines 50-105 · raises CalibrationDataError

Read and validate a calibration JSONL file line by line.

1. Fail with `CalibrationDataError` if the file does not exist.
2. For each line (1-based numbering): count it; skip blank lines; `json.loads` (invalid JSON -> counted and reported with column); require a JSON object; require both `human` and `response`; require both to be non-empty strings.
3. Valid lines become `CalibrationRecord(human.strip(), response.strip(), line_number)`.
4. Stop as soon as `max_samples` records are collected (`truncated_at_max_samples=True`).
5. Log a warning with up to 20 problem messages if anything was skipped; raise if zero valid records.

class TokenizedPair

lines 109-114 · dataclass

One teacher-forced example: encoder ids and decoder labels (no padding).

Field	Type	Default
`example_id`	int	
`input_ids`	list[int]	
`labels`	list[int]	

tokenize_calibration

```
def tokenize_calibration(records: list[CalibrationRecord], tokenizer: PreTrainedTokenizerBase, max_source_length: int, max_target_length: int) -> list[TokenizedPair]
```

lines 117-145

Tokenize `human` as encoder input and `response` as decoder labels.

1. Encoder ids: `tokenizer(human).input_ids` (includes EOS for T5 tokenizers).
2. Labels: `tokenizer(text_target=response).input_ids`.
3. If longer than the limit, keep the first `limit - 1` tokens and append EOS so sequences always terminate.
4. Return `TokenizedPair(example_id=i, input_ids, labels)`; counts of truncated sources/targets are logged.

split_train_validation

```
def split_train_validation(items: list, validation_fraction: float, seed: int) -> tuple[list, list]
```

lines 148-160

Deterministic shuffle-split; at least one item in each part when possible.

1. Shuffle the index list with `random.Random(seed)`.
2. Validation size = `max(1, round(n * fraction))` (0 if only one item).
3. The first `n_val` shuffled indices form the validation set; original order is preserved inside each part.

class PreparedCalibration

lines 164-179 · dataclass

Tokenized calibration data with its deterministic train/validation split.

Field	Type	Default
`items`	list[TokenizedPair]	
`train_ids`	set[int]	
`validation_ids`	set[int]	
`load_report`	CalibrationLoadReport	
`metadata`	dict	

`PreparedCalibration.train`

```
def train(self) -> list[TokenizedPair]
```

lines 174-175

Examples of the training split.

`PreparedCalibration.validation`

```
def validation(self) -> list[TokenizedPair]
```

lines 178-179

Examples of the validation split.

`prepare_calibration`

```
def prepare_calibration(cfg, tokenizer: PreTrainedTokenizerBase, task) -> PreparedCalibration
```

lines 182-212 · settings BIAS_VALIDATION_FRACTION, CALIBRATION_DATASET_PATH, CALIBRATION_MAX_SAMPLES, MAX_SOURCE_LENGTH, MAX_TARGET_LENGTH, SEED

Load, tokenize (with the model's task objective) and split the calibration set.

1. Load records (respecting `CALIBRATION_MAX_SAMPLES`), tokenize with `task.tokenize_calibration`, split.
2. Metadata: path, SHA-256 of the file, sizes, split seed and fraction, length limits, task name, number of input tokens and number of labelled target tokens.
3. `fingerprint` = 16-hex SHA-256 of the metadata without the path, so moving the file does not invalidate artifacts but changing content or settings does.

Module constants

Name	Value
`REQUIRED_FIELDS`	('human', 'response')

Errors raised

Function	Exception	Message (f-string fields in braces)
`load_calibration_records`	CalibrationDataError	Calibration dataset not found at {path}. Set CALIBRATION_DATASET_PATH in .env.
`load_calibration_records`	CalibrationDataError	No valid calibration records in {path}